DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS
College of Arts and Sciences | University of the Philippines Visayas
Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020
Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

A UNIX Developer Environment

CMSC 125 Lab 0

Prepared by: Rene Andre Bedonia Jocsing

Published: January 26, 2026 **Deadline:** January 30, 2026

CMSC 125 is often the first deep-dive of a CS student into UNIX-like systems, shell scripting, and systems programming. Like in CMSC 131, the instructor has taken the steps necessary to arm you for the laboratory portion of the course. The goal of this activity is to prepare a developer environment for C programming on Windows using Windows Subsystem for Linux (WSL) and Visual Studio Code.

If you already use a native Linux distribution or macOS, you may skip WSL installation and proceed directly to verifying your C toolchain for the course. However, this manual focuses on Windows users (which is 99% of the class), since WSL setup often requires careful configuration.

WSL allows you to run a genuine Linux distribution directly on Windows without dual-booting or virtual machines. This environment will be essential for understanding concepts throughout the course.

CMSC 125 has a steep learning curve—though not as steep as the one in CMSC 131?—because it introduces systems-level programming and POSIX/UNIX concepts distinct from higher-level application development. Therefore, studying the course materials ahead of time is encouraged. Again, like in CMSC 131, the concepts and assignments in this course cannot be crammed overnight (beware of one-item open-notes exams).

Installing Windows Subsystem for Linux

1. Ensure you are running Windows 10 version 2004 or higher, or Windows 11.
2. Ensure that **Virtual Machine Platform**, **Windows Hypervisor Platform**, and **Windows Subsystem for Linux** are all enabled under **Windows Features** (protip: search!).
3. Open **PowerShell** or **Command Prompt** as **Administrator**.
4. Run the following command to install WSL with Ubuntu as the default distribution:

```
wsl --install
```

5. Restart your computer, if prompted.
6. After a (possible) restart, Ubuntu will automatically launch and complete installation.
7. Create a UNIX username and password when prompted. **Remember these credentials**.
8. To verify installation, open a fresh command line and input the following:

```
wsl --list --verbose
```

You should see output showing Ubuntu with version 2 (WSL 2).

For detailed installation instructions or troubleshooting, consult the official WSL installation guide.

Setting Up the C Development Toolchain

1. Open your WSL terminal (search for "Ubuntu" in the Start menu).
2. Update the package manager:

```
sudo apt update
```

3. Install essential build tools:

```
sudo apt install build-essential gdb
```

   This installs `gcc` (GNU C Compiler), `g++`, `make`, and `gdb` (GNU Debugger).

4. Verify GCC installation:

```
gcc --version
```

   You should see version information for GCC.

Integrating Visual Studio Code with WSL

1. Download and install **Visual Studio Code** for Windows from the official website.
2. Open Visual Studio Code.
3. Under the **Extensions** tab, search for `WSL`.
4. Install the **WSL** extension by Microsoft.
5. Install the **C/C++** extension by Microsoft.
6. Press `Ctrl + Shift + P` to open the command palette.
7. Type `WSL: Connect to WSL` and press Enter.
8. Visual Studio Code will reopen connected to your WSL environment.
9. Alternatively, you can try to navigate to your working folder through Windows File Explorer, open WSL in the directory, then type `code .` If `code .` is not recognized, ensure VS Code was added to your Windows PATH during installation.
10. Visual Studio Code will open connected to the WSL environment with the directory as its working folder.

    You can verify this by checking the bottom-left corner of VS Code, which should display `WSL: Ubuntu`.

Creating Your First C Program in WSL

1. In VS Code connected to WSL, create a new folder for your projects:

```
mkdir ~/cmsc125
cd ~/cmsc125
```

2. Open this folder in VS Code: **File > Open Folder**, then select `cmsc125`.
3. Create a new file called `hello.c`.
4. Copy and paste the code from the listing below.
5. Save the file.
6. Open the integrated terminal in VS Code (‘Ctrl + “).
7. Compile the program:

```
gcc -o hello hello.c -Wall
```

8. Run the executable:

```
./hello
```

You should see `Hello, UNIX!` printed to the terminal.

```c
/*
 * file: hello.c
 * A simple C program to verify your UNIX development environment.
 */

#include <stdio.h>

int main(void) {
    printf("Hello, UNIX!\n");
    return 0;
}
```

Understanding the Build Process

Let us examine what the GCC command does:

The `gcc` command invokes the GNU C Compiler to **compile** your C source file (`hello.c`). The `-o hello` flag specifies the output filename for the executable. Without this flag, GCC would create an executable named `a.out` by default.

Behind the scenes, GCC performs four stages:

1. **Preprocessing**: Expands macros and includes header files
2. **Compilation**: Translates C code to assembly language
3. **Assembly**: Converts assembly to machine code (object files)
4. **Linking**: Combines object files and libraries into an executable

You can observe individual stages using flags like `-E` (preprocessing only), `-S` (compilation to assembly), or `-c` (compilation to object file without linking). The `-Wall` flag tells the compiler to show all warnings, which is standard for systems programming.

For more complex projects with multiple source files, you will use `make` and `Makefiles` to automate the build process. You may explore this in future laboratories.

Submission

For this activity, you must document your complete setup process:

1. Take a **screenshot** showing your WSL terminal (or other terminal as applicable) with the output of:
   - `wsl --list --verbose` (only if using WSL)
   - `gcc --version`
   - `uname -a` (only if using Linux or WSL)
2. Take a **screenshot** of VS Code connected to WSL (showing the `WSL: Ubuntu` indicator in the bottom-left corner). Again, only if using WSL.
3. Take a **screenshot** of your `hello.c` program running successfully in the terminal.
4. Create a simple `reflection.txt` containing:
   - A brief reflection on your experience setting up WSL or your C toolchain
   - Any challenges encountered and how you resolved them
   - Do you want to be a systems programmer after you graduate?
5. Zip everything into a single archive and submit through email or submission bin.

   Adhere to the following convention.

   ZIP filename: `surname_initials_lab0.zip` (e.g., `sanchez_sm_lab0.zip`)

   Subject Line: `[CMSC 125 Lab] Lab 0: Surname, Initials` (e.g., `[CMSC 125 Lab] Lab 0: Sanchez, SM`)

Deadline

No laboratory defense is required for this laboratory activity, but you must submit the deliverables **on or before January 30, 2026**. Late submissions will only **receive a maximum grade of 60%.** In this course, it is your responsibility to keep abreast of coursework and deadlines. The instructor will not remind you of any pending deliverables in the course, and will simply give you a failing grade (or, if applicable, an INC) for any missing deliverables by the end of classes.